MEDICAL RECORD QUESTION-ANSWERING SYSTEM

RAG Project Guide: Complete Workflow, Decision-Tree Diagrams & Master Prompt

A practical project document for building and evaluating two comparable RAG pipelines:

Version 1: Original embeddings (without PCA)
Version 2: PCA-reduced embeddings

Prepared as a single reference document

Use this guide alongside the official internship task brief and the supplied synthetic medical-record PDF. The task brief remains the authority if any requirement differs from this planning guide.

Document contents

- Part 1 — Complete project workflow and architecture
- Part 2 — Decision-tree workflow diagrams
- Part 3 — Evaluation plan, metrics and fair-comparison rules
- Part 4 — Suggested repository structure
- Part 5 — Master prompt to give ChatGPT / a coding assistant

Project inputs referred to in this guide

- **Dataset:** Synthetic_Medical_Record_Exercise_Whitfield.pdf
- **Official instructions:** Intern Task Brief: Medical PDF Question-Answering System (RAG).pdf

Before implementation, inspect both files and confirm page count, extractability, required models, vector database, LLM, interface and deliverables from the actual brief.

PART 1 — Complete project workflow

The project builds a medical-record question-answering system that retrieves relevant passages from the supplied PDF and generates answers grounded in those passages. It then compares two versions: one using the original embedding vectors and one using PCA-reduced vectors.

1.1 Overall project stages

1. Inspect inputs

Read the complete PDF dataset and official task brief. Identify record types, page structure, tables, dates, IDs and any extraction problems.

2. Extract PDF content

Extract text page by page. Preserve page number and any available record identifiers. If pages are scanned or text extraction is unusable, use OCR only where necessary and verify the OCR output.

3. Clean and structure

Remove extraction noise without deleting clinically meaningful wording, abbreviations, values or negations. Preserve source metadata for citations.

4. Create evaluation questions

Prepare realistic questions that can be answered from the supplied records. Create relevance labels linking each question to the record/chunk IDs that genuinely support it. Manually verify labels.

5. Run chunking experiments

Compare fixed-size, sentence-based and recursive chunking at target sizes of 300, 600 and 900 tokens, subject to tokenizer and framework constraints.

6. Select and freeze chunking

Evaluate retrieval for each chunking configuration. Select the best-supported configuration and freeze it before the PCA comparison.

7. Create original embeddings

Use BAAI/bge-small-en-v1.5 (verify actual model output dimension and its current model-card instructions). Encode all frozen document chunks.

8. Build Version 1

Store original vectors in a dedicated vector index. Embed each query in the original embedding space, retrieve Top-K chunks and pass selected context to the same RAG answer-generation setup.

9. Build Version 2

Fit PCA on document embeddings only. Reduce document vectors to 128 components if mathematically feasible. Transform both documents and query vectors with the same fitted PCA model. Normalize transformed vectors when using cosine similarity.

10. Evaluate both versions

Run the same questions through both versions with identical chunking, embedding model, Top-K settings, query rewriting/reranking settings, LLM and generation parameters.

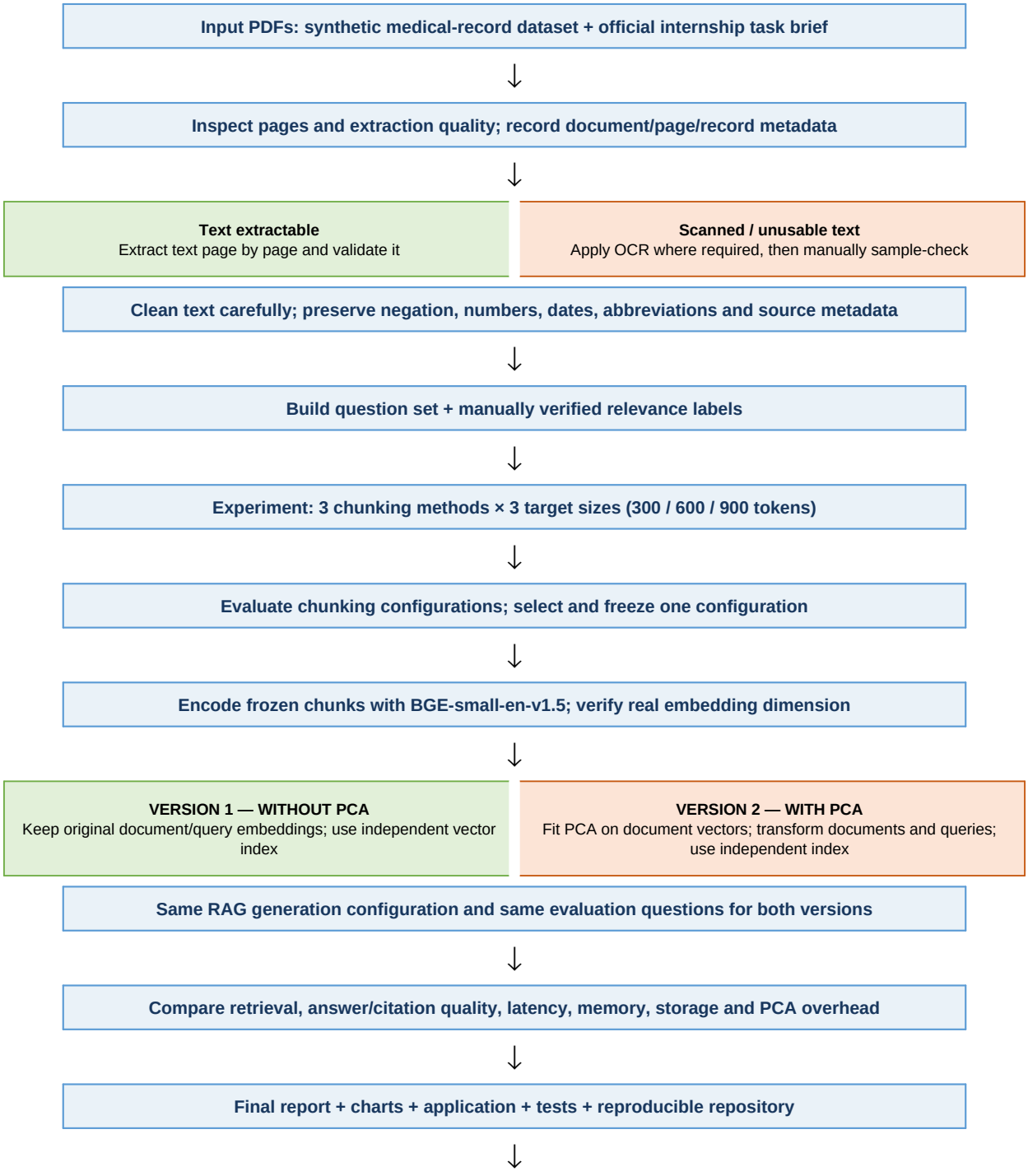
11. Compare results

Compare retrieval quality, answer quality, citations, latency, memory, vector/index size, PCA fit/transform time and explained variance. Report any quality loss as well as efficiency gains.

12. Deliver

Create a reproducible app or interface, tests, result tables/plots, documentation and a final report. Do not fabricate metrics or claim experiments that were not run.

1.2 Architecture: shared preparation, then two branches



1.3 Version 1 — Without PCA

- Embed each selected document chunk using the chosen BGE model.
- Store the full original vectors in a separate vector database/index.
- At query time, embed the user question with the same model and compatible query instruction, if required by the verified model documentation.
- Apply the same similarity metric consistently. If using cosine similarity, ensure the vector store or application computes it correctly.

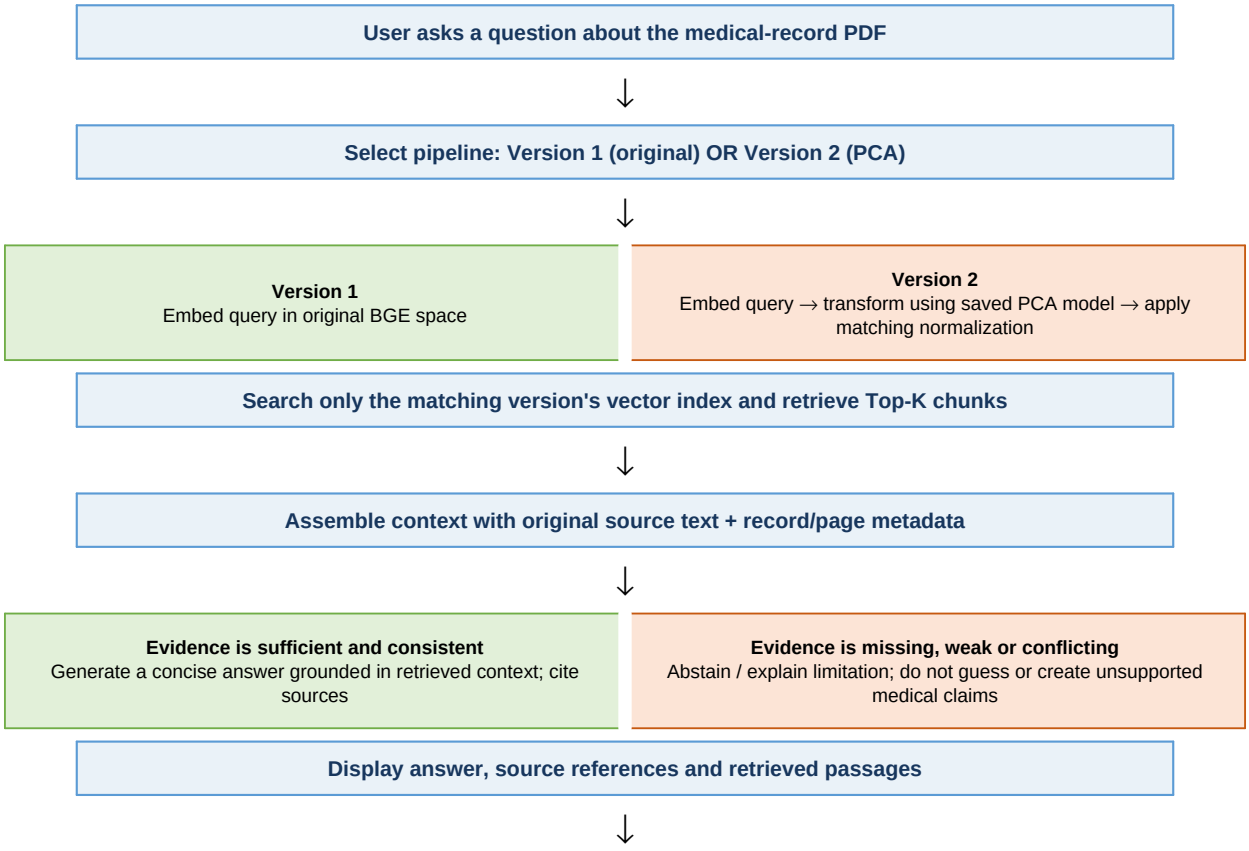
- Retrieve configurable Top-K values: 1, 3, 5, 8, 10 and 15.
- Pass retrieved source text and metadata to the configured LLM. Require the answer to use only retrieved evidence and cite record/page/chunk sources.
- If evidence is insufficient or conflicting, the system should say so rather than inventing an answer.

1.4 Version 2 — With PCA

- Start from the same original document embeddings used by Version 1.
- Fit PCA on the document embedding matrix only; do not fit PCA using query vectors or relevance labels.
- Use 128 components only if n_components is no greater than min(number of document vectors, original embedding dimension). If not feasible, choose a valid dimension and clearly document the constraint.
- Save the fitted PCA model and use that exact model to transform both document embeddings and future query embeddings.
- Use a separate vector index for PCA vectors; never mix original-space vectors with PCA-space vectors.
- If cosine similarity is used, L2-normalize the transformed document and query vectors consistently. Record the normalization choice.
- Measure PCA fit time, transform time, explained variance, reduced vector size, index size and retrieval latency.

Important: PCA is not guaranteed to improve retrieval. It may reduce storage and computation, but it can also discard dimensions that contain useful semantic information. The experiment must measure both benefits and losses.

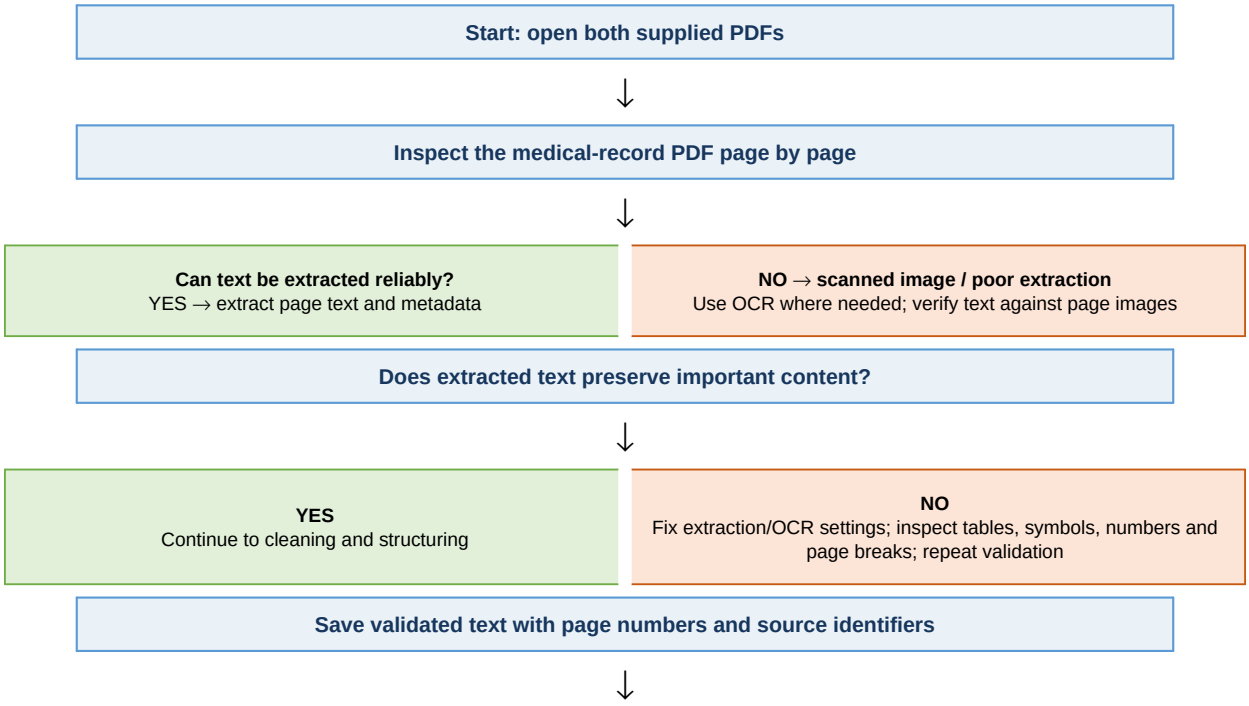
1.5 RAG query-time workflow



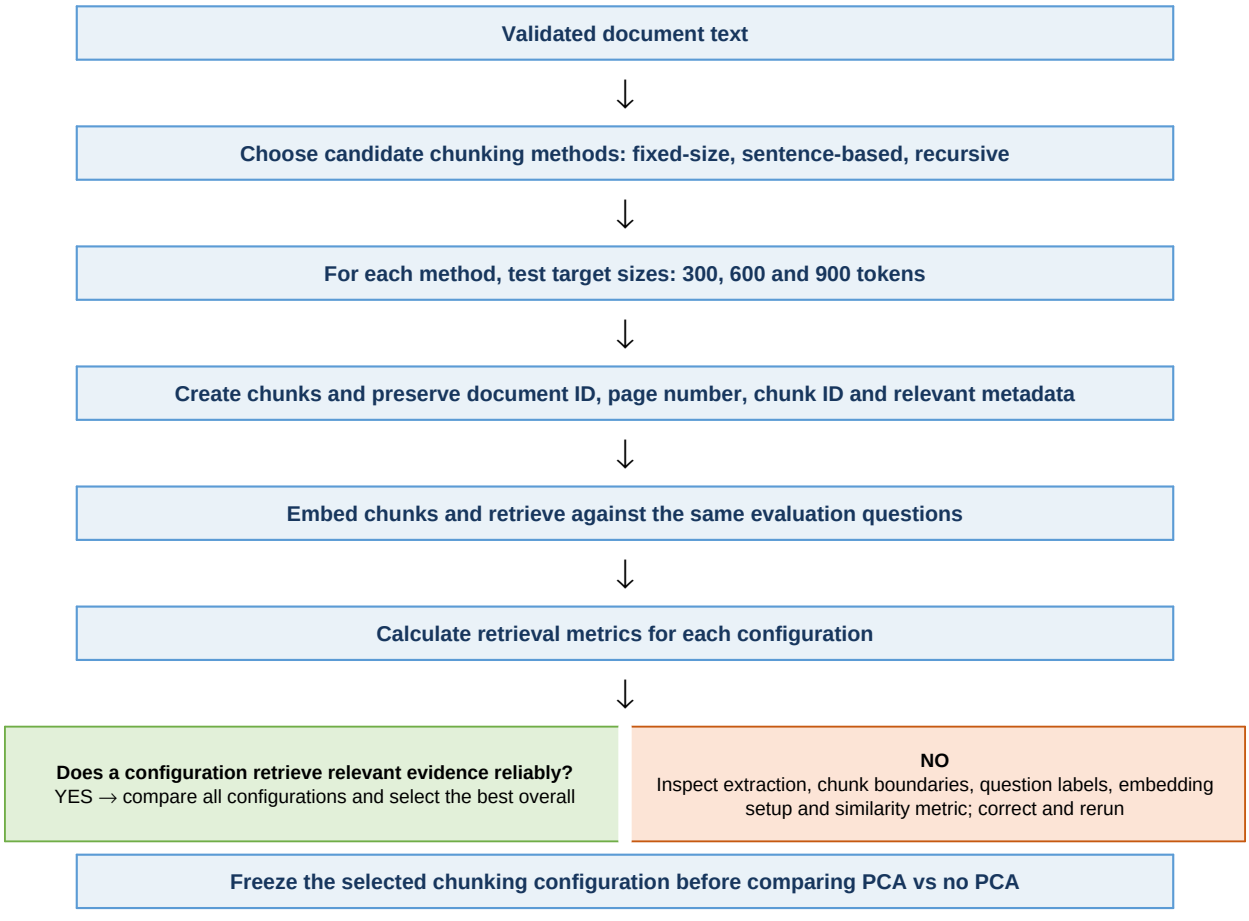
PART 2 — Decision-tree workflow diagrams

These diagrams are designed to explain the project during a review or viva. Follow the branches in order; a failed check loops back to the relevant correction stage.

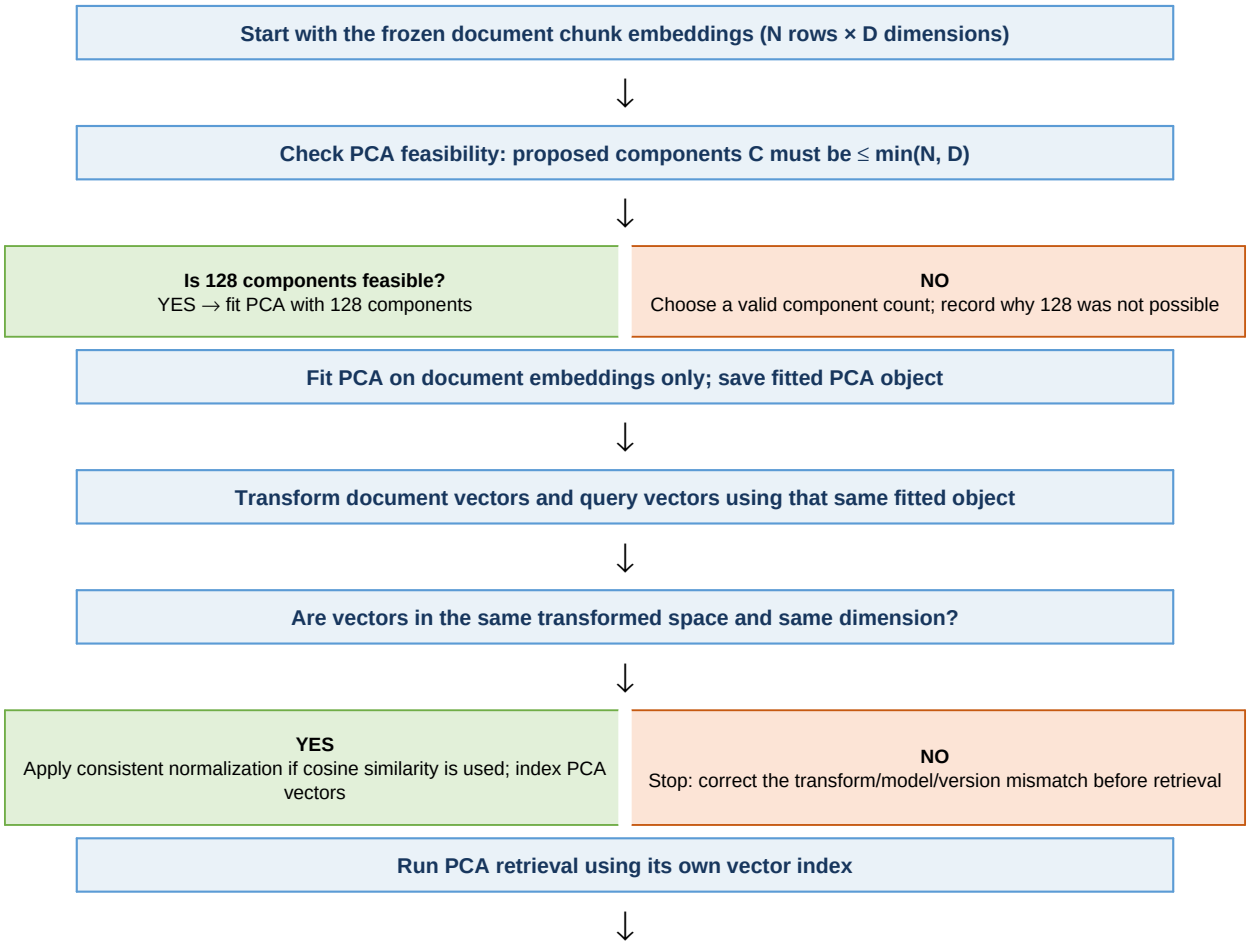
2.1 PDF extraction decision tree



2.2 Chunking experiment decision tree



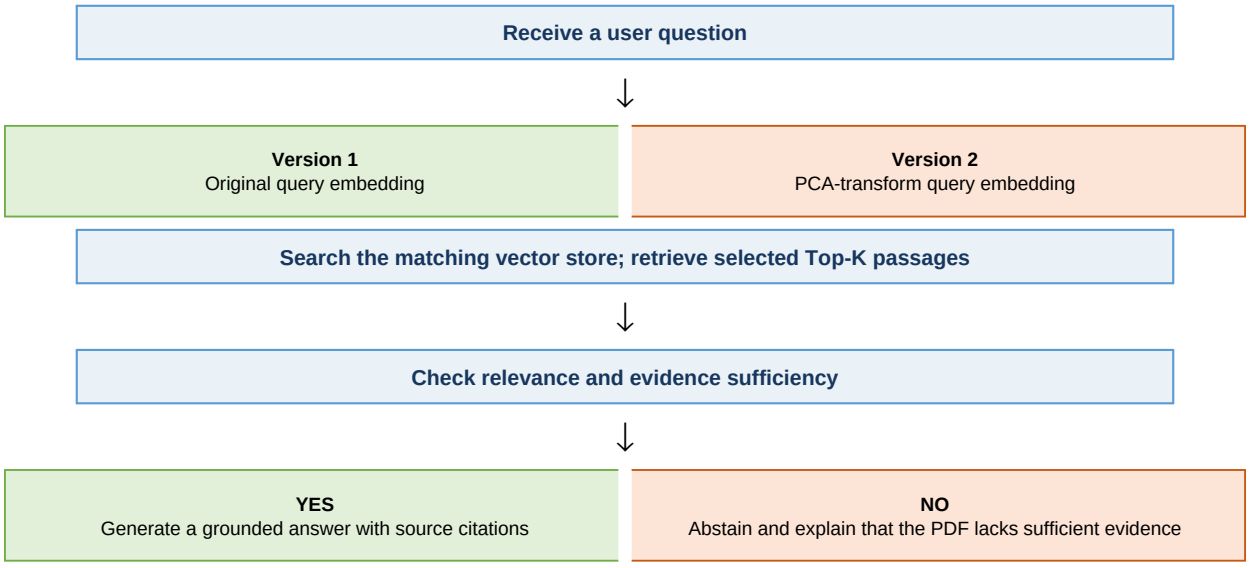
2.3 PCA feasibility and implementation decision tree



2.4 Runtime application decision tree



2.4 Runtime application decision tree



Show answer + record/page references + retrieved source text

↓

2.5 Final evaluation decision tree

Run the same evaluation questions through both versions

↓

Compare retrieval metrics: Hit Rate@K, Precision@K, Recall@K, MRR and nDCG

↓

Compare RAG outputs: correctness, faithfulness, relevance and citation correctness

↓

Compare efficiency: retrieval latency, memory, vector/index size, PCA fit and transform time

↓

Are retrieval and answer quality acceptably maintained after PCA?

↓

YES
Determine whether storage/latency savings are meaningful; report trade-off

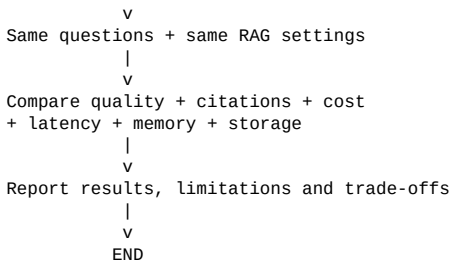
NO
Inspect explained variance and dimensions; test another valid PCA size; report quality loss honestly

Prepare tables, plots, limitations, reproducibility details and final conclusion

↓

2.6 Compact end-to-end decision tree

```
START
  |
  v
Inspect task brief + medical PDF
  |
  v
Can text be extracted?
  |-- No --> OCR + validate --> repeat extraction check
  |
 Yes
  |
  v
Clean text + preserve page/record metadata
  |
  v
Create evaluation questions + verified relevance labels
  |
  v
Chunking experiments:
  fixed / sentence / recursive × 300 / 600 / 900
  |
  v
Evaluate retrieval --> select and FREEZE best chunking
  |
  v
Create BGE document embeddings
  |
  +--------------------------------+
  |                                |
  v                                v
VERSION 1                        VERSION 2
NO PCA                           PCA
Original vectors                 Fit PCA on document vectors
Original vector index            Transform docs + queries
Original query embedding         Reduced vector index
  |                                |
  +---------------+----------------+
                  |
```


v
Same questions + same RAG settings
|
v
Compare quality + citations + cost
+ latency + memory + storage
|
v
Report results, limitations and trade-offs
|
v
END

PART 3 — Evaluation plan and fair-comparison rules

3.1 Retrieval metrics

Metric	What it tells you
Hit Rate@K	Whether at least one relevant passage appears in the top K results.
Precision@K	How many of the top K retrieved passages are relevant.
Recall@K	How much of the known relevant evidence is retrieved in the top K.
MRR	How high the first relevant result appears; higher is better.
nDCG	Ranking quality that can account for graded relevance; higher is better.

3.2 RAG answer and safety evaluation

- **Answer correctness:** Is the answer supported by the source record and factually consistent with it?
- **Faithfulness:** Does the answer avoid claims that are not supported by retrieved passages?
- **Answer relevance:** Does it answer the actual question?
- **Citation correctness:** Do the cited record/page/chunk references support the statements they accompany?
- **Abstention:** Does the system decline to answer when evidence is missing or contradictory?
- **Medical safety:** Does it avoid generating diagnoses, treatment instructions or unsupported medical advice? Keep the system focused on summarizing/retrieving existing record content.

3.3 Efficiency and PCA measurements

- Original embedding dimension and PCA output dimension.
- PCA explained variance ratio and cumulative explained variance.
- PCA fitting time and document/query transform time.
- Embedding/index construction time.
- Warm retrieval latency: repeat queries and report median and spread, not only one timing.
- Peak or measured memory usage, where practical.
- Raw vector bytes and persisted vector-index/database size.
- Total storage including the PCA model and any metadata needed at runtime.

3.4 Fair comparison checklist

- Same source PDF and same extracted/cleaned text.
- Same chunking method, target size, overlap and frozen chunk set.
- Same BGE model and model revision.
- Same evaluation questions and relevance labels.
- Same Top-K values and retrieval metric.
- Same query rewriting setting in both versions.
- Same reranking model and setting in both versions, or disable it in both.
- Same LLM, prompt, temperature and generation settings.
- Same hardware/runtime conditions as far as possible; record any differences.

- Separate vector indexes for original and PCA spaces.
- No fitting PCA on query data or evaluation labels.
- Do not invent or estimate results; report actual measured outputs.

A useful comparison is paired: each evaluation question is run through both versions, then the per-question results are compared. Include confidence intervals or paired statistical tests/effect sizes when the sample size and metric type make them appropriate. Explain limitations if the evaluation set is small.

3.5 Important limitations to disclose

- A single synthetic PDF is a limited test corpus; results may not generalize to real hospital records.
- Manual relevance labels can contain judgement errors; describe how they were created and checked.
- PCA is a linear dimensionality-reduction method and may remove useful information.
- The best PCA dimension is an empirical choice, not automatically 128.
- A smaller vector dimension does not guarantee a faster complete RAG system; measure the whole pipeline.
- Model instructions, embedding dimensions and software APIs can change; record package versions and model revision.

PART 4 — Suggested project repository structure

Use this as a clean starting structure. Adjust names to match the actual task brief and implementation, but keep the two versions clearly separated.

```
medical-record-rag-pca-evaluation/
|
+-- data/
|   +-- raw/
|   |   +-- Synthetic_Medical_Record_Exercise_Whitfield.pdf
|   +-- processed/
|   |   +-- extracted_pages.json
|   |   +-- document_chunks.json
|   +-- evaluation/
|       +-- evaluation_questions.csv
|       +-- relevance_labels.csv
|
+-- common/
|   +-- pdf_extraction.py
|   +-- preprocessing.py
|   +-- chunking.py
|   +-- embeddings.py
|   +-- evaluation.py
|   +-- config.py
|
+-- version_1_without_pca/
|   +-- src/
|   |   +-- vector_store.py
|   |   +-- retrieval.py
|   |   +-- rag_pipeline.py
|   +-- experiments/
|   +-- results/
|   +-- app.py
|   +-- run.py
|   +-- README.md
|
+-- version_2_with_pca/
|   +-- src/
|   |   +-- pca_transform.py
|   |   +-- vector_store.py
|   |   +-- retrieval.py
|   |   +-- rag_pipeline.py
|   +-- experiments/
|   +-- results/
|   +-- app.py
|   +-- run.py
|   +-- README.md
|
+-- comparison/
|   +-- compare_results.py
|   +-- statistical_analysis.py
|   +-- generate_report.py
|   +-- figures/
|   +-- tables/
```

```
|
+-- tests/
+-- docs/
|   +-- project_workflow.md
|   +-- final_project_report.md
+-- .env.example
+-- .gitignore
+-- requirements.txt
+-- README.md
```

Never commit API keys, passwords, private credentials or data that you are not authorized to publish. Check whether the supplied PDF may be included in a public GitHub repository before uploading it.

PART 5 — Master prompt for ChatGPT / coding assistant

Copy the prompt below into a new ChatGPT conversation or a coding assistant. Upload both the dataset PDF and the official internship task brief. The prompt asks the assistant to inspect the source files first, then build a complete reproducible project as files rather than giving disconnected code snippets.

ROLE
Act as a senior AI/ML engineer, Python developer, RAG architect and technical project mentor. Help me build a complete, reproducible Medical PDF Question-Answering System using Retrieval-Augmented Generation (RAG), and compare two independent implementations: one without PCA and one with PCA.

PROJECT INPUTS
I will provide two PDF files:
1. Synthetic_Medical_Record_Exercise_Whitfield.pdf — the dataset.
2. Intern Task Brief: Medical PDF Question-Answering System (RAG).pdf — the official requirements.

SOURCE-FIRST RULE
Before writing code, inspect and understand BOTH PDFs completely. Extract the official requirements, deliverables, constraints, evaluation criteria and expected technologies from the task brief. Inspect the dataset structure, page count, text extraction quality, record types and metadata. Do not assume that the dataset contains fields or structures that you have not verified. Do not silently replace an official requirement with a preferred alternative. If a requirement is unclear or conflicts with this plan, explain the conflict and follow the official brief.

MY OBJECTIVE
Build two versions using the same source PDF and the same evaluation setup:
- VERSION 1 — WITHOUT PCA: use the original BGE embedding vectors.
- VERSION 2 — WITH PCA: reduce the document/query embedding vectors using a fitted PCA model and use a separate vector index.

The purpose is to evaluate whether PCA reduces dimensionality, storage or retrieval cost while maintaining acceptable retrieval and RAG answer quality.

REQUIRED WORKING STYLE
1. Give me an ordered end-to-end workflow and explain the purpose of each stage in beginner-friendly but technically accurate language.
2. Create the complete project as actual files in the workspace. Do not give only isolated code snippets.
3. Use modular Python files, clear configuration, reusable functions, logging, error handling and comments.
4. Make reasonable engineering decisions, but explain them and record them in the documentation.
5. Do not claim that a command, test, experiment or result succeeded unless you actually ran it and observed the result.
6. If something fails, diagnose it, fix it and rerun the relevant test.
7. Keep the implementation reproducible with fixed random seeds where applicable and record package/model versions.
8. Do not expose secrets. Use environment variables and provide a .env.example file with placeholder values only.
9. Do not upload the supplied medical PDF or any sensitive data to a public repository unless I explicitly confirm that it is permitted.
10. If a required external model or API is unavailable, implement a clearly documented local/retrieval-only fallback where feasible; do not pretend generation has been evaluated if it has not.

PHASE A — UNDERSTAND THE SOURCE FILES
- Read both PDFs.
- Summarize the task brief requirements in a requirement checklist.
- Summarize the dataset structure based only on what is actually present.
- Check whether the PDF is text-based or scanned. Use OCR only where necessary.
- Preserve page numbers, record identifiers and other useful metadata.
- Identify any missing information or ambiguity and ask me only essential questions.
- Do not start implementation until the source requirements and plan are clear.

PHASE B — PROJECT DESIGN AND DATA PREPARATION
- Propose a clean repository structure.
- Extract text page by page.
- Clean extraction noise carefully without removing meaningful medical terms, negations, values, dates, abbreviations or units.
- Preserve source metadata so the application can show the original record/page.
- Save intermediate outputs in reproducible formats such as JSON or CSV.
- Validate extraction with sample pages and document known extraction limitations.

PHASE C — CHUNKING EXPERIMENTS
Compare these three chunking strategies:
1. Fixed-size chunking.
2. Sentence-based chunking.
3. Recursive character/text chunking.

Test target chunk sizes of 300, 600 and 900 tokens, subject to the actual tokenizer and library constraints. Use a documented overlap strategy and keep it consistent where applicable. Preserve document ID, page number, chunk ID and source text for every chunk.

Create an evaluation question set and relevance labels. The relevant record/chunk labels must be grounded in the PDF and manually verified. Do not fabricate labels. Evaluate the chunking configurations using retrieval metrics. Select the best-performing and most defensible configuration, explain why, and FREEZE it before the PCA comparison. Do not change chunking between Version 1 and Version 2.

PHASE D — EMBEDDINGS
Use BAAI/bge-small-en-v1.5 as the planned embedding model unless the official brief requires a different model. Verify the model card, correct query/document encoding conventions, actual output dimension, model revision and package compatibility before implementation. Do not assume the output dimension without checking it.

Generate embeddings for the frozen document chunks. Save or cache embeddings reproducibly so they do not need to be regenerated unnecessarily. Apply the model's documented query instruction/prefix correctly, if applicable, and use the same convention in both versions.

PHASE E — VERSION 1: WITHOUT PCA
- Use the original embedding vectors without dimensionality reduction.
- Build a dedicated vector store/index using a technology compatible with the official brief (for example, ChromaDB, FAISS or another required option).
- Implement configurable similarity search.
- Evaluate Top-K values: 1, 3, 5, 8, 10 and 15.
- Return retrieved chunk text with record/page/source metadata and similarity score.
- Implement the RAG pipeline using the LLM required or permitted by the task brief. Prefer a suitable free/local option if allowed.
- The generated answer must be grounded in retrieved context and must cite source record/page references.
- If the evidence is insufficient or conflicting, the system must abstain or clearly state the limitation.

PHASE F — VERSION 2: WITH PCA
- Start from the same original document embedding matrix used in Version 1.
- Fit PCA on DOCUMENT embeddings only. Do not fit PCA on query embeddings, relevance labels or test answers.
- Target 128 principal components only if mathematically valid: n_components must not exceed min(number of document vectors, original embedding dimension). If 128 is not feasible, select a valid component count and explain the constraint.
- Save the fitted PCA model and its configuration.
- Transform document embeddings and query embeddings with the same fitted PCA model.
- Keep a separate vector store/index for PCA-reduced vectors.
- If cosine similarity is used, normalize transformed document and query vectors consistently after transformation, and document this choice.
- Measure explained variance, PCA fit time, transform time, reduced vector size, index size, memory and retrieval latency.
- Never mix original-space vectors with PCA-space vectors.

PHASE G — RAG AND APPLICATION
- Implement a clean query workflow: user question → selected version → query embedding (and PCA transform if Version 2) → matching vector index → Top-K retrieval → evidence sufficiency check → grounded answer with citations.
- Include query rewriting as a configurable option if it is allowed and useful under the task brief. Apply the same setting to both versions during comparison.
- Include reranking as an optional configurable stage if allowed and useful. If enabled for the comparison, use the same reranker and settings in both versions.
- Use the same LLM, system prompt, temperature and generation settings for both versions.
- Keep the answer focused on information present in the supplied records. Do not generate diagnoses, treatment advice or unsupported medical conclusions.
- Build the user interface required by the brief; if no UI is specified, a simple Streamlit interface is acceptable.
- Show answer, source references and retrieved passages. Include a way to select Version 1 or Version 2 and configure Top-K.

PHASE H — EVALUATION
Use exactly the same evaluation questions, relevance labels, frozen chunks, embedding model, query rewriting/reranking settings, LLM and generation settings for both versions.

Retrieval metrics:
- Hit Rate@K.
- Precision@K.
- Recall@K.
- Mean Reciprocal Rank (MRR).
- Normalized Discounted Cumulative Gain (nDCG), if graded relevance labels are available.

RAG answer metrics:
- Answer correctness.
- Faithfulness to retrieved context.
- Answer relevance.
- Citation correctness.
- Appropriate abstention when evidence is insufficient.

Efficiency metrics:
- Embedding dimension.
- PCA explained variance and cumulative explained variance.
- PCA fitting and transformation time.
- Index construction time.
- Retrieval latency (repeat runs; report median and spread).
- Memory usage where measurable.
- Raw vector storage and persisted index/database size.
- Include PCA model storage/overhead when comparing total storage.

Use paired per-question comparisons because both systems answer the same questions. Use confidence intervals and suitable paired statistical tests/effect sizes when the dataset size and metric type permit. Explain small-sample limitations. Do not invent, estimate or fabricate experimental results.

PHASE I — VISUALIZATION AND REPORTING
Create useful tables and plots comparing:
- Retrieval metrics across Top-K.
- Answer-quality and citation results.
- Original vs reduced embedding dimensions.
- PCA explained variance.
- Retrieval latency and index/build time.
- Memory and storage.
- Quality-versus-efficiency trade-offs.

Write a professional final report with:
1. Executive summary.
2. Problem statement and objectives.

3. Source data and task requirements.
4. Architecture and workflow.
5. PDF extraction and preprocessing.
6. Chunking experiments and selection rationale.
7. Embedding model and vector-store configuration.
8. Version 1 implementation.
9. Version 2 PCA implementation.
10. Evaluation methodology and metrics.
11. Results and comparison.
12. Limitations, risks and safety.
13. Conclusion and future improvements.
14. Reproduction instructions.

PHASE J — TESTING AND FINAL DELIVERABLES

- Add tests for extraction, chunk metadata, embedding dimensions, PCA fit/transform consistency, vector-store isolation, retrieval ranking, citation metadata and abstention behaviour.
- Run the project end to end on the supplied dataset.
- Run tests and fix errors.
- Provide complete runnable source files, requirements.txt, .env.example, .gitignore, README files and final report.
- Save actual experiment outputs, CSV result tables and plots.
- Provide a ZIP archive of the complete project if the environment supports it.
- Give exact setup and execution commands.
- Clearly list what was implemented, what was tested, actual results, known limitations and any remaining manual steps.

SUGGESTED REPOSITORY

medical-record-rag-pca-evaluation/
data/raw/
data/processed/
data/evaluation/
common/
version_1_without_pca/src/
version_1_without_pca/experiments/
version_1_without_pca/results/
version_2_with_pca/src/
version_2_with_pca/experiments/
version_2_with_pca/results/
comparison/figures/
comparison/tables/
tests/
docs/
requirements.txt
.env.example
.gitignore
README.md

FINAL RULE

The final comparison must be scientifically fair: the only planned difference between the two retrieval branches is whether PCA dimensionality reduction is applied. Keep all other relevant settings identical. If a difference is unavoidable, disclose it and explain its effect. Never manufacture results. Build the complete project, verify it, and then provide the deliverables and exact run instructions.